Tests Folder – System Documentation

Oct 10, 2026 · @Kai-Kristian

Overview

The folder `/home/kai/tests` holds a small website uptime monitor, not a conventional test suite. Its package name is `website-uptime-monitor`, version 1.0.0. One run opens a target website in Chromium through Playwright, measures the result, and writes one row to a PostgreSQL table.

The default target is `https://www.lyrikblog.at`. The code is TypeScript and runs directly with `ts-node`. The folder also contains the unmodified Playwright sample spec, which is not part of the monitor.

A single run goes through these steps:

1. `check-website.ts` loads the configuration and decides between headless and visible browser mode.
2. `UptimeMonitor.run()` creates the table `tests.website_checks` if it does not exist.
3. `WebsiteChecker` launches Chromium, opens the target URL and waits for `domcontentloaded`.
4. The checker returns a result: online flag, HTTP status code, response time in ms, error message, timestamp.
5. `DatabaseWriter` inserts the result as one row.
6. Browser and database pool are closed, and the result is printed to the console.

Directory structure

Eleven files are tracked in git; five further entries exist only locally.

Path	In git	Purpose
`check-website.ts`	Yes	Entry point. Reads the config, runs one check, prints the result.
`config/params.ts`	Yes	Loads `.env` and exports the typed `config` object.

Path	In git	Purpose
`src/UptimeMonitor.ts`	Yes	Orchestrates one run: init table, check, write, clean up.
`src/WebsiteChecker.ts`	Yes	Playwright wrapper that opens the URL and builds a `CheckResult`.
`src/DatabaseWriter.ts`	Yes	PostgreSQL pool, table creation and insert.
`sql/create-table.sql`	Yes	Manual DDL for `tests.website_checks`, same as the code creates.
`example.spec.ts`	Yes	Playwright sample spec against playwright.dev.
`package.json`	Yes	Package metadata, scripts and dependencies.
`tsconfig.json`	Yes	TypeScript compiler settings, output to `dist/`.
`.env.example`	Yes	Template for the environment file.
`readme.txt`	Yes	Placeholder with one line of text.
`.env`	No	Local environment values, loaded at runtime.
`project.env`	No	Second environment file with the same variable names. No code loads it.
`package-lock.json`	No	npm lockfile.
`node_modules/`	No	Installed packages.
`test-results/`	No	Playwright output. Holds only `.last-run.json`.

Tooling and dependencies

The monitor needs Node.js, a Chromium build installed by Playwright, and a reachable PostgreSQL server. Version ranges below come from `package.json`.

Package	Range	Kind	Used for
`playwright`	^1.44.0	dev	Launches Chromium in `WebsiteChecker`.
`@playwright/test`	^1.44.0	dev	Test runner for `example.spec.ts`.
`pg`	^8.12.0	runtime	PostgreSQL client and connection pool.

Package	Range	Kind	Used for
`dotenv`	^16.6.1	runtime	Loads `.env` into `process.env`.
`ts-node`	^10.9.2	dev	Runs the TypeScript entry point directly.
`typescript`	^5.4.5	dev	Compiler for the `build` script.
`@types/node`	^20.19.43	dev	Node type definitions.
`@types/pg`	^8.11.6	dev	Type definitions for `pg`.

TypeScript compiles to ES2020 CommonJS with `strict` enabled. The compiler includes `src/`, `config/` and `check-website.ts`, and excludes all `*.spec.ts` files.

Configuration and environment

All settings come from environment variables, read by `config/params.ts` from the `.env` file in the folder root. Only `DB_PASSWORD` is mandatory; the run stops with an error if it is missing.

Variable	Default in code	Meaning
`TARGET_URL`	`https://www.lyrikblog.at`	Website to check.
`REQUEST_TIMEOUT_MS`	15000	Playwright navigation timeout in ms.
`HEADLESS`	`true`	Any value other than `false` runs the browser headless.
`DB_HOST`	`localhost`	PostgreSQL host.
`DB_PORT`	5432	PostgreSQL port.
`DB_NAME`	`kdb`	Database name.
`DB_USER`	`appuser`	Database user.
`DB_PASSWORD`	none, required	Database password.
`DB_CONNECT_TIMEOUT_MS`	5000	Connection timeout in ms.

The command line overrides `HEADLESS`: `--no-headless` shows the browser, `--headless` hides it. If both flags are given, `--no-headless` wins.

`.env.example` sets different values than the code defaults for `HEADLESS`, `DB_HOST` and `DB_NAME`. Values in the local `.env` and `project.env` are not reproduced here.

Components and data model

Three classes carry the monitor, and one spec file holds the only Playwright tests.

Component	File	Responsibility
`UptimeMonitor`	`src/UptimeMonitor.ts`	`run()` initialises the table, opens the browser, checks, writes, then closes browser and pool in a `finally` block.
`WebsiteChecker`	`src/WebsiteChecker.ts`	`open()`, `check()`, `close()`. Opens a new page per check and returns a `CheckResult`.
`DatabaseWriter`	`src/DatabaseWriter.ts`	`initialize()` creates the table, `writeResult()` inserts one row, `close()` ends the pool.

How a check decides the result:

- The status code is taken from the response whose URL equals the target URL, with or without a trailing slash.
- A status code below 400 counts as online. If no matching response is seen but navigation succeeds, the site also counts as online and the status code stays empty.
- A navigation error or timeout counts as offline, and the error text is stored.
- Response time is the time from the start of navigation to `domcontentloaded`, or to the error.

The sample spec `example.spec.ts` has two tests against `https://playwright.dev/`: one checks the page title, one follows the "Get started" link to the Installation heading. It does not test the monitor.

Results go to the table `tests.website_checks`:

Column	Type	Null	Content
`id`	SERIAL, primary key	No	Row id.
`check_date`	DATE	No	Date of the check, taken from the UTC ISO timestamp.
`check_hour`	SMALLINT	No	Hour of the check in server local time.
`check_timestamp`	TIMESTAMPTZ	No	Exact time of the check.
`url`	VARCHAR(255)	No	Checked URL.
`is_online`	BOOLEAN	No	Result of the check.
`status_code`	SMALLINT	Yes	HTTP status of the main response.
`response_time_ms`	INTEGER	Yes	Measured time in ms.
`error_message`	TEXT	Yes	Error text when the check failed.

Running it

One-time setup:

1. Run `npm install` in the folder.
2. Run `npx playwright install chromium` to download the browser.
3. Copy `.env.example` to `.env` and set the database values, at least `DB_PASSWORD`.
4. Make sure the schema `tests` exists in the target database. The code creates the table but not the schema.

Command	Effect
`npm run check`	Runs one check with `ts-node` and writes one row.
`npm run check -- --no-headless`	Same, with a visible browser window.
`npm run build`	Compiles to `dist/` with `tsc`.
`npm run check:build`	Compiles, then runs `node dist/check-website.js`.

Command	Effect
`npx playwright test`	Runs `example.spec.ts`. No npm script is defined for it.

A check prints mode, URL, status (ONLINE or OFFLINE), HTTP code, time in ms, date and hour. The process exits with code 1 only when the run itself fails, for example on a database error. An offline website still ends with exit code 0.

Each run performs exactly one check. No scheduler is defined in the folder, so repeated checks need an external trigger such as cron.

`test-results/.last-run.json` records the last Playwright run as `failed` with an empty list of failed tests.

Maintenance notes

The repository is on branch `main` with three commits, the latest titled "modified 13/09/2026". These points were found while reading the code and are not yet fixed:

- **No `.gitignore`.** `.env`, `project.env`, `node_modules/` and `test-results/` are untracked only by chance and could be committed by accident.
- **Password in the template.** The tracked `.env.example` contains a filled-in `DB_PASSWORD` value and an internal host name. If the value is real, it should be rotated and replaced with a placeholder.
- **Compiled run cannot find `.env`.** `config/params.ts` resolves `.env` relative to its own directory. After `tsc` that is `dist/.env`, so `npm run check:build` fails on the missing `DB_PASSWORD` unless the variables are exported in the shell.
- **Date and hour use different time zones.** `check_date` comes from the UTC timestamp, `check_hour` from local time. Around midnight the two do not match.
- **Missing status counts as online.** If the site redirects to a different URL, no response matches the target and the check reports online with an empty status code.
- **Schema not created.** The table DDL assumes the schema `tests` exists. The sample query in `sql/create-table.sql` omits the schema prefix.
- **Cleanup gap.** If table creation or browser launch fails, `run()` leaves before its `try` block and the database pool is not closed.
- **No Playwright config and no monitor tests.** There is no `playwright.config.ts`, and the only spec is the vendor sample.

- **Unused files.** `project.env` is loaded by nothing, and `readme.txt` holds a placeholder line.